

Smart Patio Shield - CNN & Fusion (Colab)

Engine-room notebook for all GPU work: Model 2 (satellite CNN) and Model 3 (multimodal fusion).

How to use: run Section 0 once per session, then run top-to-bottom.

- **0** Session setup (Drive mount, symlinks, sanity check)
- **1** Build both branches (tabular + vision inference)
- **2** Out-of-fold stacking (Priority 1)
- **3** Intensity-conditional fusion (Priority 2)
- **4** Save results

Drive layout expected:

```
MyDrive/smart-patio-shield/  
  data/          patio_features.parquet, image_labels.parquet  
  data/goes/     goes_YYYY-MM-DD.npz  
  models/        xgboost_baseline.json, cnn_resnet18_ir_visible.pt, ...
```

Section 0 - Session setup

Run once per session. Idempotent: removes stale links and rebuilds them, so a fresh runtime always ends clean. No manual symlink management.

```
In [ ]: import os, torch  
print("GPU:", torch.cuda.get_device_name(0) if torch.cuda.is_available()  
      else "NONE - Runtime > Change runtime type > T4 GPU")  
  
if not os.path.exists("/content/smart-patio-shield"):  
    from getpass import getpass  
    tok = getpass("GitHub token: ")  
    !git clone https://{tok}@github.com/romayneg/smart-patio-shield.git /content/smart-patio-shield  
    %cd /content/smart-patio-shield
```

```
!git pull -q
print("code up to date")
```

```
In [ ]: from google.colab import drive
drive.mount('/content/drive')
DRIVE = "/content/drive/MyDrive/smart-patio-shield"
```

```
In [ ]: import os, shutil

def relink(target, linkname):
    if os.path.islink(linkname):      os.unlink(linkname)
    elif os.path.isdir(linkname):     shutil.rmtree(linkname)
    elif os.path.exists(linkname):    os.remove(linkname)
    parent = os.path.dirname(linkname)
    if parent:
        os.makedirs(parent, exist_ok=True)
    os.symlink(target, linkname)

relink(f"{DRIVE}/data/goes",          "data/raw/goes")
relink(f"{DRIVE}/data/image_labels.parquet", "data/processed/image_labels.parquet")
relink(f"{DRIVE}/data/patio_features.parquet", "data/processed/patio_features.parquet")
relink(f"{DRIVE}/models",             "models")
print("symlinks created")
```

```
In [ ]: checks = {
    "data/raw/goes": os.path.isdir("data/raw/goes"),
    "image_labels.parquet": os.path.exists("data/processed/image_labels.parquet"),
    "patio_features.parquet": os.path.exists("data/processed/patio_features.parquet"),
    "xgboost_baseline.json": os.path.exists("models/xgboost_baseline.json"),
    "cnn_resnet18_ir_visible.pt": os.path.exists("models/cnn_resnet18_ir_visible.pt"),
}
for k, v in checks.items():
    print(f" { 'OK ' if v else 'MISSING' } {k}")
print("day-files:", len(os.listdir("data/raw/goes")))
assert all(checks.values()), "A path did not resolve – check Drive layout above."
print("Setup complete.")
```

Section 1 - Build the two branches

Loads Model 1 (XGBoost, CPU) and Model 2 (ResNet-18, GPU) and runs each over train/val/test, returning probabilities and the CNN's 512-d embeddings. `tab`, `img_*`, `E_*` defined here are used by Sections 2-3. Vision inference over ~87k patches takes a few minutes.

```
In [ ]: import importlib, gc, torch
import src.models.goes_dataset as gd
import src.models.cnn as cnn
import src.models.fusion as fus
for m in (gd, cnn, fus): importlib.reload(m)

tab = fus.tabular_branch("models/xgboost_baseline.json",
                        "models/baseline_training.manifest.json")
print("tabular rows - val/test:", len(tab['val'][0]), len(tab['test'][0]))

In [ ]: patches = gd._load_all_patches()
train_ds = gd.GoesPatchDataset("train", channels=gd.ALL_BANDS, patches=patches)
NORM = train_ds.norm_stats
val_ds = gd.GoesPatchDataset("val", channels=gd.ALL_BANDS, norm_stats=NORM, patches=patches)
test_ds = gd.GoesPatchDataset("test", channels=gd.ALL_BANDS, norm_stats=NORM, patches=patches)
print("train-only norm stats:", NORM)

In [ ]: model = cnn.build_model(3)
model.load_state_dict(torch.load(f"{DRIVE}/models/cnn_resnet18_ir_visible.pt"))

img_train, E_train = fus.vision_branch(model, train_ds)
img_val, E_val = fus.vision_branch(model, val_ds)
img_test, E_test = fus.vision_branch(model, test_ds)
print("vision rows - train/val/test:", len(img_train), len(img_val), len(img_test))
del patches; gc.collect()
```

Section 2 - Out-of-fold stacking

Fix: the original meta-learner was fit on ~13k validation probabilities from base models early-stopped on that same set, i.e., biased and small. Out-of-fold predictions (each row predicted by a model that never saw it) give ~51k unbiased rows.

Documented caveat: the CNN's train-hour predictions are still in-sample (no k-fold CNN retrain). So the tabular side is honest, the vision side optimistic - an improvement, not a full fix.

```
In [ ]: oof_tab, oof_X = fus.oof_tabular_predictions(
        "models/xgboost_baseline.json", "models/baseline_training.manifest.json")
print(f"OOF rows: {len(oof_tab):,}")

train_al = fus.align(oof_tab, oof_X, img_train, E_train)
val_al = fus.align(tab['val'][0], tab['val'][1], img_val, E_val)
test_al = fus.align(tab['test'][0], tab['test'][1], img_test, E_test)
print(f"aligned - train {len(train_al['y']):,} | val {len(val_al['y']):,} | test {len(test_al['y']):,}")

In [ ]: p_oof_late, meta_oof = fus.late_fusion(train_al, test_al)
print("=== TEST - out-of-fold stacking ===")
fus.report("Model 1 (tabular)", test_al["y"], test_al["p_tab"])
fus.report("Model 2 (vision)", test_al["y"], test_al["p_img"])
fus.report("Model 3a late fusion (OOF-fitted)", test_al["y"], p_oof_late)
print("meta coef [tab, img]:", meta_oof.coef_.round(4), " intercept:", meta_oof.intercept_.round(4))
```

Section 3 - Intensity-conditional fusion

Idea: satellite IR only beats ERA5 for heavier convective rain ($> \sim 1\text{-}2$ mm/h). A single global weight on `p_img` can't exploit that. We add interaction terms `p_img × context`, where context = ERA5 signals associated with convective intensity (humidity, low cloud), letting the meta-learner trust vision more in those conditions.

Row order is matched to each aligned split by looking the aligned keys back up in the branch frames - no positional assumptions.

```
In [ ]: import numpy as np
from sklearn.linear_model import LogisticRegression
from sklearn.preprocessing import StandardScaler

CONTEXT = ["relative_humidity_2m", "cloud_cover_low"] # must exist in tabular schema

def context_rows(al, tab_frame, tab_X):
    """Return tab_X rows in the SAME order as aligned dict `al` (match on key)."""
    pos = {k: i for i, k in enumerate(tab_frame['key'].to_numpy())}
    idx = [pos[k] for k in al['keys']]
    return tab_X.iloc[idx].reset_index(drop=True)

def build_Z(al, ctx_df, means=None, stds=None):
    base = np.column_stack([al['p_tab'], al['p_img']])
```

```

inter = []
for c in CONTEXT:
    if c in ctx_df.columns:
        v = ctx_df[c].to_numpy(dtype=float)
        inter.append((c, v))
return base, inter

# training context (standardization stats come from TRAIN only)
ctx_tr = context_rows(train_al, oof_tab, oof_X)
ctx_te = context_rows(test_al, tab['test'][0], tab['test'][1])

def assemble(al, ctx_df, stats=None):
    base = np.column_stack([al['p_tab'], al['p_img']])
    cols = [base]
    out_stats = {}
    for c in CONTEXT:
        if c in ctx_df.columns:
            v = ctx_df[c].to_numpy(dtype=float)
            if stats is None:
                mu, sd = np.nanmean(v), np.nanstd(v) + 1e-9
                out_stats[c] = (mu, sd)
            else:
                mu, sd = stats[c]
            vz = (v - mu) / sd
            cols.append((al['p_img'] * vz).reshape(-1, 1))
    return np.hstack(cols), (out_stats if stats is None else stats)

Ztr, stats = assemble(train_al, ctx_tr)
Zte, _ = assemble(test_al, ctx_te, stats)
print("Z shapes - train:", Ztr.shape, "test:", Zte.shape)

scaler = StandardScaler().fit(Ztr)
meta_c = LogisticRegression(class_weight="balanced", max_iter=2000, random_state=42)
meta_c.fit(scaler.transform(Ztr), train_al['y'])
p_cond = meta_c.predict_proba(scaler.transform(Zte))[:, 1]

print("\n=== TEST - intensity-conditional fusion ===")
fus.report("Model 1 (tabular)", test_al['y'], test_al['p_tab'])
fus.report("Model 3a late fusion (OOF)", test_al['y'], p_oof_late)
fus.report("Model 3c intensity-conditional fusion", test_al['y'], p_cond)
order = ["p_tab", "p_img"] + [f"p_img*{c}" for c in CONTEXT]
print("coef:", dict(zip(order, meta_c.coef_[0].round(4))))

```

Section 4 - Save results

Writes the new numbers to `model3_fusion_results_v2.json` on Drive - the receipt the documentation cites.

```
In [ ]: import json
        from datetime import datetime, timezone
        from sklearn.metrics import average_precision_score, confusion_matrix

        def block(y, p, thr=0.5):
            ap = average_precision_score(y, p)
            tn, fp, fn, tp = confusion_matrix(y, (p >= thr).astype(int)).ravel()
            return {"pr_auc": round(float(ap), 4), "recall": round(tp/(tp+fn), 4),
                    "precision": round(tp/(tp+fp), 4),
                    "confusion_matrix": {"tn": int(tn), "fp": int(fp), "fn": int(fn), "tp": int(tp)}}

        results = {
            "created_utc": datetime.now(timezone.utc).isoformat(),
            "evaluation": "held-out test, modality intersection",
            "n_test": int(len(test_al['y'])),
            "note": ("OOF stacking + intensity-conditional fusion. OOF fixes the biased/undersized "
                    "meta-learner set; conditional fusion operationalizes the satellite's >2mm/h "
                    "advantage. CNN train-hour predictions remain in-sample (documented caveat)."),
            "models": {
                "model1_tabular": block(test_al['y'], test_al['p_tab']),
                "model2_vision": block(test_al['y'], test_al['p_img']),
                "model3a_late_fusion_oof": block(test_al['y'], p_oof_late),
                "model3c_conditional_fusion": block(test_al['y'], p_cond),
            },
            "oof_meta_coef": meta_oof.coef_.round(4).tolist(),
            "cond_meta_coef": meta_c.coef_.round(4).tolist(),
            "cond_feature_order": ["p_tab", "p_img"] + [f"p_img*{c}" for c in CONTEXT],
        }
        out = f"{DRIVE}/models/model3_fusion_results_v2.json"
        json.dump(results, open(out, "w"), indent=2)
        print("saved:", out)
        print(json.dumps(results["models"], indent=2))
```